A reproducible workflow for cross-layer diagnosis of mobile robot motion pauses

Abstract

Intermittent mobile robot pauses can coincide with queue accumulation and avoidance-state changes, making attribution from one signal unreliable. We present a reproducible archival workflow that aligns observations, tests a specified explanation, audits field semantics and records where attribution must stop. Its worked case comprises four real-platform runs crossing Robot Operating System 2 (ROS 2) and Python interfaces with avoidance disabled and enabled. Each condition has one run in fixed order; four earlier C++ sessions provide separate descriptive evidence. A pause required absolute reported longitudinal velocity below 0.015 m/s for more than 0.35 s after a 2 s startup exclusion. Primary counts were zero, three, zero and four. All seven intervals followed avoidance-flag rises while requests continued; 33 within-leg timing rotations associated at most three. All 1,304 primary commands received successful acknowledgments, and active State-message gaps remained below 60 ms despite growing receive queues. The specified progression from queue growth to stale feedback was therefore not observed. Inspection of archived artifacts identified a default identifier, lost point-height information and a 500 ms release hold, but equivalence to the historical executing controller is unestablished. A separate 95-file audit illustrates incomplete control-state coverage and timestamp limitations. Executable evidence reports assign no physical root cause to any incident session. The contribution is a documented workflow and its reproducible execution on a bounded case, rather than established causal fault isolation or validated diagnostic performance.

Keywords: mobile robot; motion pause; cross-layer diagnosis; ROS 2; runtime monitoring; telemetry semantics; retrospective case study

1 Introduction

Mobile robot behavior emerges from application software, middleware, device interfaces, platform control and physical interaction. Modular architectures enable deployment across varied settings,[1] but they also distribute the evidence needed to explain an unexpected stop. A command may be issued on time and acknowledged successfully while the platform deliberately suppresses motion. Conversely, an accumulating socket queue may indicate a real reliability defect without explaining the current motion symptom. Confusing these cases can lead to a repair at the wrong layer.

Execution monitoring has long distinguished detecting an execution deviation from identifying its cause.[2] Software-fault corpora also show that robotic failures arise from varied implementation and integration problems.[3] Runtime verification[4,5] and tracing[6,7] provide ways to inspect these systems. However, field records are often collected to resolve an engineering incident rather than to evaluate a new diagnostic algorithm. Such records can support a useful empirical study if the observation chain, comparison units and unresolved alternatives are made explicit.

We examine a historical incident in which a mobile base intermittently approached zero longitudinal velocity during constant-speed manual motion. Two explanations were plausible: accumulating command acknowledgments might eventually interrupt feedback, or platform avoidance control might intervene despite a continuous upstream command. The available archive contains a four-run comparison crossing ROS 2 and Python entry paths with avoidance disabled and enabled. It includes network captures and socket observations alongside motion records, allowing the two explanations to be assessed on the same timeline. Earlier C++ sessions and archived deployed artifacts provide complementary checks on recurrence and on the meaning of reported control fields.

RQ1 asks what the archive establishes at each observation layer. **RQ2** asks which conclusions persist under entry-path contrasts, event definitions and timing perturbations. **RQ3** asks where attribution must stop when field semantics, historical executing versions and triggering inputs are incomplete. We contribute an explicit workflow with an input contract, executable evidence reports and a stopping rule for unsupported root attribution. Its worked case separates recorded command/feedback continuity from avoidance-state correspondence; an artifact audit identifies information lost in serialized fields. Additional archives test field availability rather than supplying matched treatment repetitions. Unlike fault-tolerant navigation research,[8] the study does not evaluate a controller or fault-isolation algorithm. Following case-study reporting principles,[9] we

treat the fixed-order records as one bounded engineering case. Workflow portability is documented, while diagnostic accuracy and transfer to other systems remain unevaluated.

2 Related work

Execution monitoring addresses whether an action proceeds as expected and how failures can be classified.[2] The distinction matters when a residual between desired and reported velocity has several possible causes. In model-based control research, Saito and Jiang[8] integrate actuator diagnosis, model switching and obstacle-aware navigation, with evaluation in simulation. Our records contain neither actuator-current measurements nor injected faults, and cannot support that type of fault-isolation or controller comparison. They instead permit comparison of several observed software and control explanations for the same symptom.

ROSMonitoring evaluates runtime properties of ROS applications.[4] Caldas et al.[5] provide guidelines for runtime verification and field testing, emphasizing the challenges of real deployment contexts. Our retrospective rules assess available observations without online enforcement or a formal coverage evaluation.

The tracing literature is particularly relevant to the evidence boundary. The ros2_tracing framework combines middleware execution information with operating-system tracing.[6] Message-flow analysis can reconstruct complex causal links in distributed ROS 2 systems.[7] Our archive has application timestamps, packet arrivals and kernel queue snapshots, but no middleware tracepoints or controller callback traces. Consequently, the temporal ordering here lacks the causal-link evidence available from such instrumentation. Historical diagnostics, as envisaged in ROS REP 107,[10] are valuable here, but a diagnostic status or successful response still requires interpretation against the motion record.

ROBUST documents software faults and failures across ROS projects.[3] It motivates keeping failure manifestations separate from fault mechanisms, but its bugs are not labeled examples for our platform. We therefore use literature to position the question and reporting method, rather than treating unrelated datasets or algorithms as directly comparable baselines. The empirical comparison in this study is between observation sets applied to the same four records.

3 Materials and methods

3.1 Archive and study design

The primary corpus comprises four consecutive archived comparison runs recorded on 17 September 2026 on one commercial mobile base. Their order was ROS 2 with avoidance off, ROS 2 with avoidance on, Python with avoidance off, and Python with avoidance on. Each condition has one run. The operator's contemporaneous notes described smooth motion in the two off runs and visible hesitation in the two on runs. These notes are corroborating observations, not blinded annotations or independent physical measurements.

The historical task commanded longitudinal velocity of −0.10 m/s toward a 1.5 m reverse target, followed by settling and forward motion toward the run's starting position at +0.10 m/s. The acquisition script required at least 2 s of low reported velocity during settling. Each run used its own odometric origin; the records do not establish a common externally measured path. The script included bounds on speed, pose and feedback age, a 45 s deadline per leg, and a stop path for exceptions. No new motion experiment or configuration change was performed. A later diagnostic file outside the event window was excluded.

Both entry paths used the same installed software development kit (SDK) library, whose recorded SHA-256 was identical across runs. ROS 2 used the platform's existing driver with TwistStamped commands and Odometry feedback through CycloneDDS in an isolated domain; Python used public drive and odometry bindings. Application updates were approximately 20 Hz, whereas the SDK emitted nonzero ManualMove WebSocket commands at approximately 10 Hz. The prebuilt ROS driver and locally inspected SDK checkout have different source provenance; a dirty engineering-build identifier cannot establish exact source equivalence. We therefore rely on captured behavior and artifact hashes rather than attributing deployed behavior to a particular current source line.

The archive records readbacks of the avoidance setting from two platform nodes at approximately 1 Hz, plus configuration and restoration checks. Off runs involved historical setting changes; they are diagnostic contrasts, not a proposed operating policy. The present analysis does not recommend disabling avoidance. Simulated harness runs are excluded. Four earlier C++ sessions are analyzed separately as secondary triangulation because their stopping rules and comparison structure differ. Files within an archive are not additional trials.

3.2 Evidence layers and provenance

We retained byte-for-byte local copies of motion CSVs, call and event logs, raw packet captures, decoded frames, socket snapshots, parameter monitors and acquisition provenance. A SHA-256 manifest binds each copy to its original local path. Re-executing the archived TCP/WebSocket decoder in a separate working directory reproduced all four decoded-frame files byte-for-byte. An additional packet-header audit checked capture truncation and advertised zero TCP windows. Capture logs reported zero kernel drops; these checks reduce specific capture concerns but cannot certify absence of every instrumentation error.

The application layer records call time, phase, target velocity and call duration. The wire layer records commanded velocity, application-level ManualMove acknowledgments and platform State frames. The kernel layer samples the command connection's Recv-Q, Send-Q and receive-memory accounting approximately five times per second. The control layer uses State.isAvoid and the contemporaneous avoidSensorList. The motion layer uses longitudinal odometry velocity and displacement. Parameter readbacks establish the sampled setting, rather than assuming that a requested setting persisted.

These layers are complementary but not all independent. Application programming interface (API) odometry and raw State velocity originate from the platform's feedback path; their agreement is a consistency check, not independent motion ground truth. Likewise, an acknowledgment captured on the network establishes a returned response, not its consumption by the SDK or successful actuation. Receive-queue payload bytes and kernel receive-memory bytes are different quantities and are reported separately.

3.3 Alignment and event definitions

Packet times and recorded wall times originate from the same host. We mapped monotonic sample times to that host's wall clock using the median wall-minus-monotonic offset in the socket observer, then expressed all times relative to the run's GO event. Offset variation was below 1 μs within each run. This internal consistency does not measure network delay, controller-clock synchronization or sensor acquisition latency. State transitions are observed at packet arrival, at roughly 51 ms intervals; event endpoints therefore have sampling uncertainty on that scale.

The primary pause definition reproduces the historical analysis. Within each reverse or forward leg, samples from the first 2 s are excluded. Consecutive samples below 0.015 m/s absolute longitudinal velocity form candidate intervals, retained if they last more than 0.35 s. Duration runs from the first eligible low-speed sample to the first subsequent sample above threshold. Planned settling is excluded. An interval ending with the leg would be flagged as censored; none of the retained intervals was censored. The rule detects sustained near-zero odometry, not a fault by itself.

Separately, we enumerated every false-to-true isAvoid transition between GO and round-trip completion, without startup exclusion. Each episode ends at its first subsequent false frame. For each transition we measured the delay to the first State frame below the near-zero threshold. We also counted nonempty avoidSensorList frames and measured the interval between the first empty list and flag release. These quantities characterize the reported sequence. We interpret identifier and point fields only after the artifact audit below; no physical sensor, true obstacle or collision threshold is inferred from field names alone.

3.4 Diagnostic workflow and robustness checks

The workflow has five steps (Table 3; Supplementary S12). First, define the symptom, time basis, whole-run units and missing fields. Second, inspect requests, replies, queues and feedback against a stated communication explanation. Here, the proposed chain requires queue accumulation to precede feedback disruption and stale feedback before stopping. Third, associate control-state transitions with reported velocity, retaining startup and censored events separately. Fourth, check field meanings against archived artifacts while recording whether the historical executing build is linked. Fifth, issue separate observation, association and artifact-interpretation statements, and leave unsupported physical attribution unresolved. These outputs are evidence reports, not fault-classifier labels.

The offline implementation maps existing outputs to a documented input dictionary and marks a dependent assessment unavailable when its observation fields are withheld. For this case, historical executing-controller equivalence, event-specific triggering inputs/decisions and independent motion measurements remain unestablished. These unresolved links stop a sensor-specific root-cause claim; they are not proposed as universally sufficient conditions for causality. An upstream command gap or an avoidance flag would still

require event-specific interpretation. The reusable component is the assessment procedure and missingness contract, not a prevalidated causal model.

Acknowledgments were paired within each command TCP connection using first-in, first-out (FIFO) ordering. The records lack unique per-request correlation identifiers, so this pairing assumes replies preserve request order. We report missing or unmatched responses, error codes and captured command-to-response intervals. This is not a measurement of server computation time. Nonzero-command intervals are calculated within each direction to avoid including the planned reversal. State continuity is assessed over GO-to-completion, and feedback age over motion samples.

We repeated pause extraction over speed thresholds of 0.010, 0.015 and 0.020 m/s and minimum durations of 0.25, 0.35 and 0.50 s. Startup exclusion was varied over 0, 1, 2 and 3 s. We also repeated the primary detector on raw State velocity and shifted sample-to-packet alignment by −100, −50, 0, +50 and +100 ms. For the alignment check, a pause remained associated if a flag rise occurred within the preceding 1 s. All choices are retrospective; none is presented as preregistered.

To test timing specificity, we also rotated all flag-rise times within their original motion legs by 33 fixed fractions from 0.10 to 0.90 in increments of 0.025, wrapping at each leg boundary. Each control used the same fraction across legs, preserving flag counts and pause windows. The association lookback remained 1 s. These dependent retrospective controls test temporal correspondence; they are not independent trials or a permutation test.

3.5 Secondary sessions and implementation audit

The secondary corpus comprises an early C++ avoidance-on probe aborted by its stationary guard, two complete avoidance-off round trips, and one complete avoidance-on round trip. The repeat off run followed an operator request to repeat a run that had not been seen. The on-run harness was changed to continue observing pauses rather than abort at the first stationary interval. These selections preclude treating the sessions as interchangeable repetitions of the primary matrix. The same event rule was reapplied, with time measured from the first reverse sample because these files have no GO event. Incomplete speed intervals and flag episodes retain their censoring status. Eight distinct capture hashes distinguish the primary and secondary sessions; re-decoding the four secondary captures reproduced their stored frames byte-for-byte.

Separately archived deployed headers, a message schema and three server libraries were inspected without executing them. A local ELF load-segment reader checked every instruction word in 18 preserved disassembly extracts against the copied binary bytes. We traced default construction, point conversion, serialization and collision-release logic using symbol names and object offsets. This validates artifact interpretation, not exact identity with the binaries executing during the historical runs: that runtime link is unavailable. We inverse-transformed each rising-edge point with the same State frame's planar pose, using x_local = cos(theta)(X−P_x) + sin(theta)(Y−P_y) and y_local = −sin(theta)(X−P_x) + cos(theta)(Y−P_y). These coordinates are a conditional reconstruction of reported points, not independent sensor measurements. We did not infer the original point height, fit a collision boundary or substitute current parameters for historical values.

The experimental unit is the whole run. The eight primary legs, seven primary intervals, nine primary flag episodes and thousands of samples are nested observations. Secondary sessions add descriptive triangulation, not independent randomized replication. With one fixed-order run per condition, independent replication and exchangeability needed for a factorial test or population confidence interval are absent. We therefore report exact counts, durations, observed ranges and within-interface descriptive contrasts. We do not bootstrap samples or events, calculate population p-values, or interpret zero observed events as a zero failure probability.

3.6 Additional archive eligibility and timing audit

A read-only roster and filesystem audit sought additional motion records. Candidate fragments and MCAP schemas/statistics were inspected within explicit access and truncation limits (Supplementary S10). Identified simulations, playback, code and conversation records were excluded as physical evidence. We selected all nine closed navigation archives from one run series on a second source unit across six calendar days. Content audits also covered 20 collection archives from one task and all 66 further summary-positive files containing ManualMoveCmd and State messages. These observational archives remain separate from the incident comparison.

For navigation, we copied indexed raw chunks containing every input and forwarded command, with neighboring odometry coverage. Decoded command counts were checked against archive summaries. Ordinal pairing compared all three planar velocity components; repeated bodies and missing correlation identifiers limit correspondence. Recording-time differences are not computation latency. Control-state availability and sustained near-straight command exposure determine whether the primary interpretation is testable. For both content cohorts, complete summary counts and first, middle and last indexed chunks distinguish declared channels from recorded messages. Fixed chunk sampling cannot characterize whole-record motion. Private ledgers retain range offsets, hashes, stability checks and decoding parameters; deidentified values reproduce the analyses.

4 Results

4.1 Four-run contrast and completeness

All four archived runs reached round-trip completion with a zero acquisition exit code. The motion CSVs contain 7,076 samples, of which 2,597 belong to the reverse or forward phases. These are logging counts, not independent trials. Table 1 summarizes the run outcomes. The return offsets are computed from the platform's own odometry and cannot establish external positioning accuracy.

Table 1. Recorded run design and odometric outcomes. Each row represents one run; no independent within-condition replication is available.

Run	Interface	Avoidance	Completion time (s)	Primary pauses	Return offset (cm)
01	ROS 2	Off	36.014	0	2.97
02	ROS 2	On	38.311	3	3.47
03	Python	Off	35.905	0	2.43
04	Python	On	39.724	4	3.03

The primary detector identified no pauses in either avoidance-off run and three and four pauses in the ROS 2 and Python avoidance-on runs, respectively. Their combined duration was 1.702 s for ROS 2 and 2.405 s for Python. Dividing by the post-startup motion exposure gives 5.83% and 7.86%, respectively; the corresponding off-run fractions were zero. These are fractions of two particular records, not estimates of population incidence. GO-to-completion time increased by 2.298 s in the ROS 2 pair and 3.818 s in the Python pair. Different motion transients and the fixed sequence prevent interpreting those differences as treatment effects.

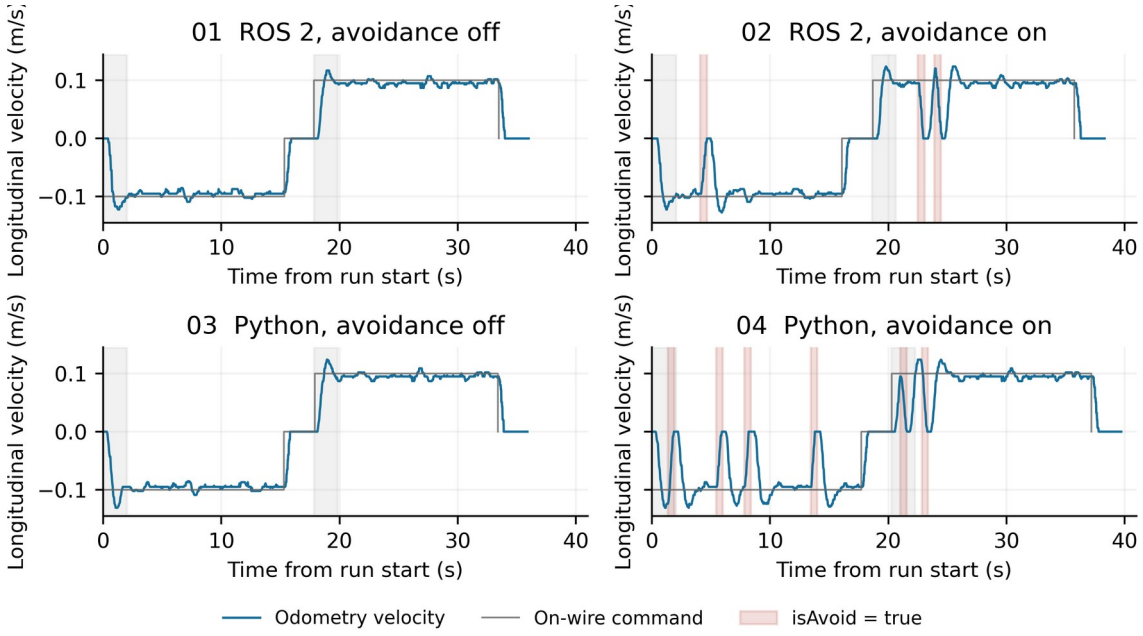


Figure 1. Four primary histories. Blue: sampled odometry; gray: on-wire command; red: isAvoid=true; pale gray: first 2 s of each leg. Planned reversal and settling remain visible but are excluded from pause detection. Both on records contain reported-velocity pauses. Each curve is one run.

4.2 Continuous commands and accumulating acknowledgments

Across the four capture sessions, 1,304 commands were paired with 1,304 successful application acknowledgments (ACKs). There were no unmatched replies, unacknowledged commands or nonzero error codes. The largest captured acknowledgment interval, 7.631 ms, occurred in the Python avoidance-off run, which had no primary pause. Application call gaps remained below 51.12 ms, and maximum nonzero wire-command gaps remained below 100.32 ms (Table 2). Nonzero commands continued in the local windows around all nine avoidance transitions.

Table 2. Communication observations. Command/ACK counts cover the capture session including stop commands; State gaps cover GO-to-completion. Maxima are observed values, not uncertainty bounds.

Run	Command / ACK	Max API gap (ms)	Max wire gap (ms)	Max ACK interval (ms)	Max State gap (ms)
01	312 / 312	51.118	100.124	2.202	59.320
02	334 / 334	50.870	100.261	3.180	59.493
03	310 / 310	50.443	100.256	7.631	58.578
04	348 / 348	50.884	100.311	3.375	59.741

Recv-Q nevertheless increased in every run, including both smooth runs (Figure 2). Session maxima ranged from 38,750 to 43,500 payload bytes. The largest observed receive-memory allocation was 73,728 B against a recorded 131,072 B budget, or 56.25%. Sampled Send-Q was always zero. The packet audit found no established command-flow zero-window advertisements or truncated captured packets. These measurements do not constrain unobserved queue behavior between snapshots or predict longer sessions.

Active State-message gaps were at most 59.741 ms across the four runs. The largest sampled motion feedback age was 150.135 ms in ROS 2 and 55.528 ms in Python, below the acquisition guard of 250 ms. Thus, queue growth was real, but the required feedback-starvation step in the proposed accumulation-to-stale-feedback chain was not observed in these records. The observations do not establish that acknowledgment accumulation is harmless over longer operation.

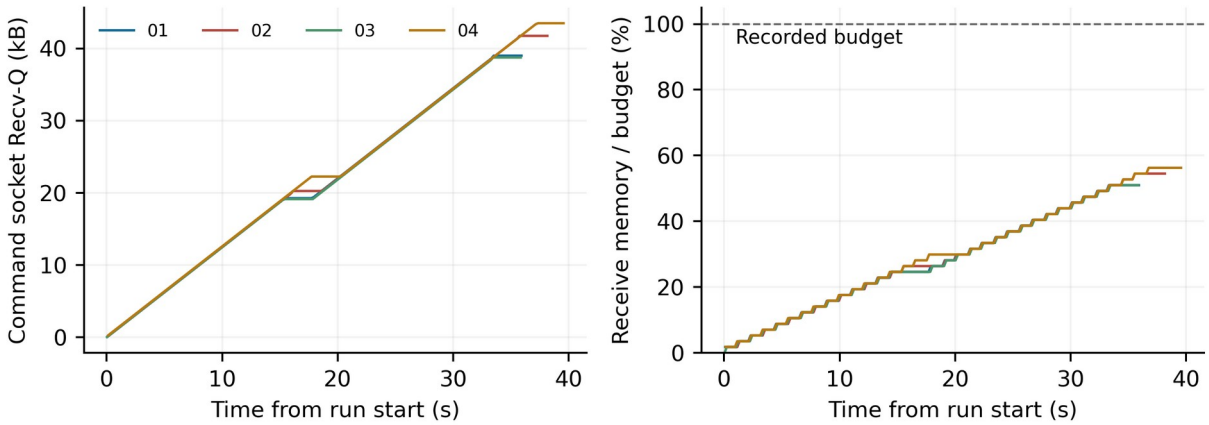


Figure 2. Command-connection queue growth in every run. Left: Recv-Q payload bytes, using decimal kB. Right: kernel receive-memory allocation divided by the recorded budget. Lines identify the four runs in Table 1; each is one time series. Queue growth also occurred in the two runs without primary pauses. The budget line is a recorded accounting limit, not a prediction of time to failure. No interpolation-based confidence bands are used.

4.3 Control transitions and velocity response

The uncensored audit found three isAvoid rising edges in the ROS 2 on run and six in the Python on run; the off runs had none. Two Python transitions occurred 1.303 and 0.750 s after their respective leg starts, explaining why the primary startup-filtered detector does not summarize all control events. All seven primary pauses followed flag rises, with the flag still true at the mapped beginning of the low-speed interval.

Across all nine transitions, Absolute State velocity first fell below 0.015 m/s after 0.308–0.471 s and subsequently reached a reported zero. Flag episodes lasted 0.565–0.618 s. Every rising-edge frame included avoidSensorList identifier 0, and each episode contained two nonempty list frames. The list became empty before flag release, with an observed interval of 0.462–0.515 s. Figure 3 displays all nine episodes. These

timings describe persistent reported status after a brief point list. Section 4.6 compares that pattern with archived artifact semantics, conditionally on an unverified historical build link.

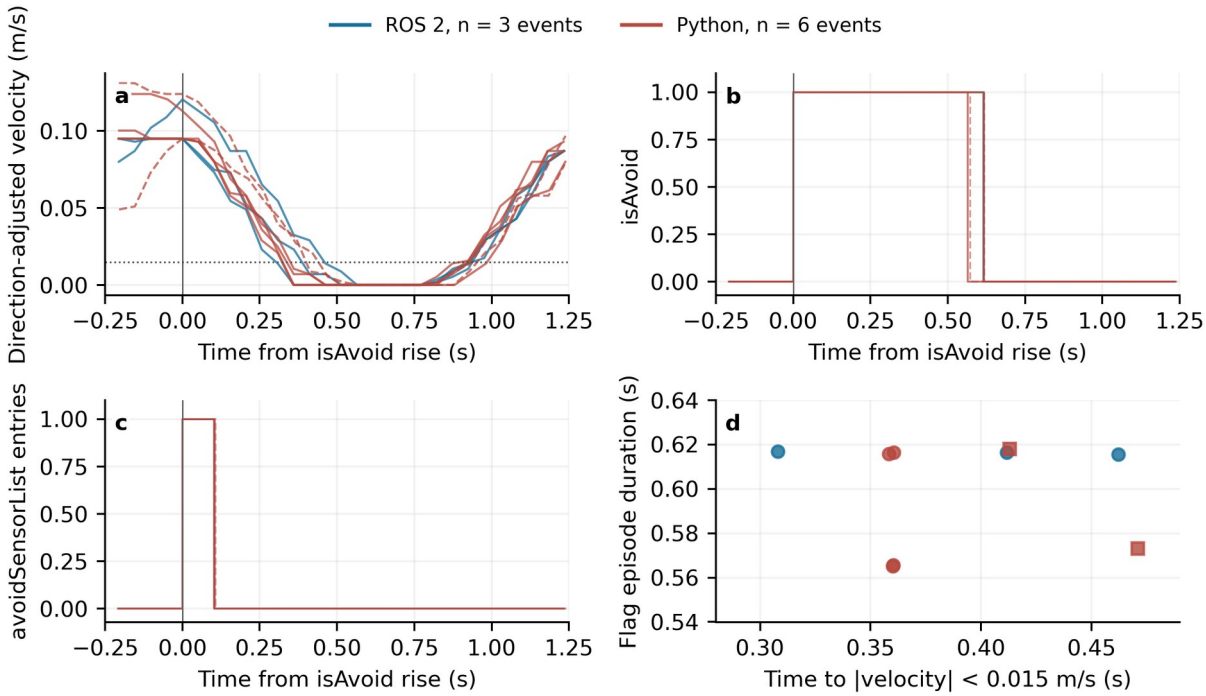


Figure 3. All nine avoidance transitions aligned at the rising edge. (a) Direction-adjusted raw State velocity; dotted line, 0.015 m/s. (b) isAvoid. (c) avoidSensorList entry count. (d) Episode duration versus delay to near-zero velocity. Blue denotes ROS 2 and red Python. Dashed trajectories and square points denote the two startup events omitted by the primary pause rule. Each curve or point is a nested event, not an independent run; overlapping curves are retained. The displayed window ends before the next rising edge.

No active State frame reported emergency stop, edge contact, explicit pause, suspension or parking through the corresponding flags. Reported navigation error codes were zero and driver-error lists were empty. Such negative diagnostic fields constrain reported explanations; they do not independently exclude mechanical faults or environmental interaction. The two-node avoidance-setting readbacks matched each run's assigned setting at all 149 sampled motion-window checks. Transient setting changes between these approximately 1 Hz observations remain unobservable.

4.4 Robustness and evidence withholding

For the 2 s startup exclusion, all nine speed–duration combinations produced zero events in both off runs. Counts ranged from two to three in the ROS 2 on run and from three to five in the Python on run (Figure 4). The contrast persisted, but counts depended on the definition. At the primary speed and duration, eliminating startup exclusion counted two low-speed intervals in each off run, showing that ordinary acceleration can satisfy the speed-duration rule. A 1 s exclusion retained six Python-on intervals, compared with four at 2 s and 3 s. The onset-period flag events therefore require separate reporting.

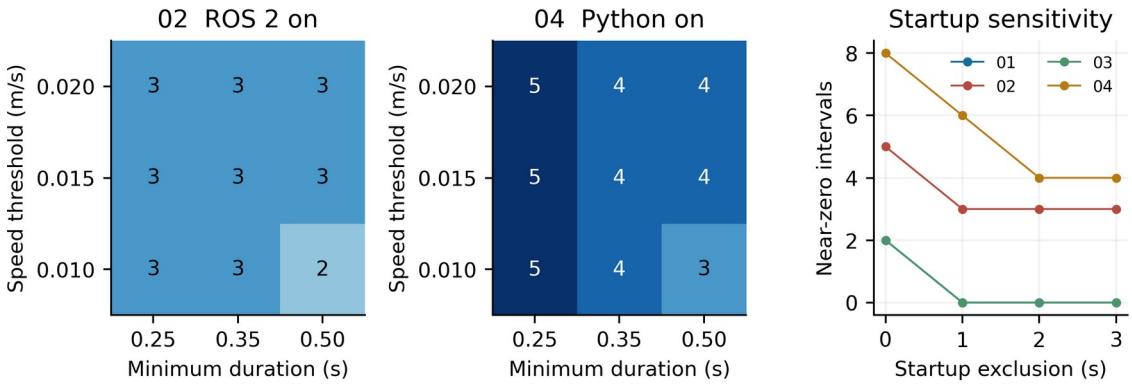


Figure 4. Event-definition sensitivity. Heatmaps show pause counts at 2 s startup exclusion for all nine speed-duration combinations in the two on runs; the off-run counts were zero throughout this grid. The right panel varies startup exclusion at the primary speed-duration pair. Zero exclusion also counts ordinary acceleration intervals in off runs. The grid is an exploratory robustness check on fixed records, not a set of new experiments.

Applying the primary detector to raw State velocity reproduced counts of zero, three, zero and four. Raw-State interval durations were 0.515–0.673 s, compared with 0.501–0.702 s in API-sampled odometry, consistent with different sampling paths. All seven primary intervals retained a preceding flag rise within 1 s under every tested alignment shift. This perturbation checks the association's tolerance to alignment error; it does not establish physical causal latency.

All seven primary pauses retained the unshifted 1 s association. The 33 within-leg rotations associated zero to three of seven, with none reproducing all seven (Supplementary Figure S2). This makes an alignment explanation based solely on flag counts less consistent with these records, without supplying a population significance test.

The workflow separates symptom detection, captured command continuity, feedback observations and control-state correspondence (Table 3). Evidence withholding identifies which statements lose their required observations (Supplementary S12). Matching readbacks constrain sampled settings but do not make the run-order contrast causal.

Table 3. Reusable workflow steps and the limits of their outputs in the worked case. Application to another platform requires new field mappings and case-specific verification.

Workflow step	Permitted assessment	What remains unresolved
1 Define the case	Declare whole-run units, clocks, symptom rules, startup exclusion and censoring.	Reported velocity is not independent physical motion truth.
2 Inspect the upstream path	Compare requests, captured replies, queues and feedback with a specified explanation.	Reply arrival does not establish consumption or actuation.
3 Associate control state	Report flag timing and matching sampled setting readbacks.	One fixed-order run per condition cannot identify a setting effect.
4 Audit field semantics	Interpret schemas and archived code, recording the historical version link.	An archived path is not proof of its execution at an event.
5 State the attribution limit	Export observations, unavailable fields and unresolved links separately.	Missing triggering inputs and motion truth prevent physical root attribution here.

4.5 Additional real-platform sessions

The two complete C++ off runs yielded no primary-rule pauses or isAvoid rises; the complete C++ on run yielded seven pauses and seven rises (Table 4). All 1,007 commands across these four secondary captures had successful replies, including 26 in the aborted session. Receive queues also grew in both complete off runs. Thus, the main distinction recurred through a third entry path. Different stopping rules, selection during troubleshooting and unmeasured environmental changes prevent pooling these sessions into a balanced interface experiment.

The aborted run ended after 2.355 s of motion with exit code 9 and about 0.127 m reverse displacement. After startup exclusion, its low-speed interval lasted at least 0.351 s and was right-censored. Two flag rises occurred before the motion record ended; only the first had an observed release within that window. The second rise

occurred while velocity was already near zero. In the complete C++ on run, one of seven rises likewise occurred at an already near-zero velocity. A flag rise therefore cannot be counted automatically as a distinct newly initiated stop. Supplementary Figure S1 retains all four secondary histories.

Table 4. Secondary C++ sessions, analyzed separately from the primary matrix. S1 ended at its stationary guard; its low interval and second flag episode are right-censored at the final motion sample. Command/ACK counts cover each full capture.

Session	Avoidance	Outcome	Primary-rule intervals	Flag rises	Command / ACK
S1	On	Aborted (exit 9)	1 (censored)	2	26 / 26
S2	Off	Complete	0	0	310 / 310
S3	Off	Complete	0	0	312 / 312
S4	On	Complete	7	7	359 / 359

4.6 Telemetry semantics and the unresolved sensor cause

All 10,303 instruction words in the 18 disassembly extracts matched the archived ELF bytes. In the audited conversion path, avoid_points is an array of points without source identifiers. Each output object is default-constructed with identifier zero and then assigned a position; serialization exports that default identifier. Thus, id=0 is not evidence of a particular physical sensor in this implementation. The planar point conversion assigns output z from pose.z, discarding original point height. All 18 rising-edge point records across the two corpora had identifier zero and z equal to the same-frame pose z, consistent with that conversion. These observations cannot establish the original sensor or input height.

The nine primary points reconstructed to x = 0.2005–0.2035 m and y = 0.2971–0.3029 m, spanning approximately 3.0 by 5.8 mm (Figure 5). The earlier C++ on points were more dispersed, including negative local x. The small primary cluster therefore characterizes this four-run sequence rather than a universal trigger location. Neither clustering nor repeated identifiers distinguish a real obstacle, self-return, extrinsic error or perception artifact without the original sensor inputs.

The archived controller header specifies a 500 ms collision hold. The corresponding binary block updates the last-collision time on a hit and retains collision status while the elapsed integer milliseconds are below 500. The point list can clear before this retained status ends. Observed release after the first empty list was 0.462–0.515 s in the nine primary episodes and 0.461–0.516 s in the seven complete C++ on episodes; the first aborted-run episode gave 0.512 s. This is consistent with release hold on the roughly 51 ms State publication scale. It does not identify a periodic trigger, prove exact internal latency or establish that this particular binary was loaded during every run.

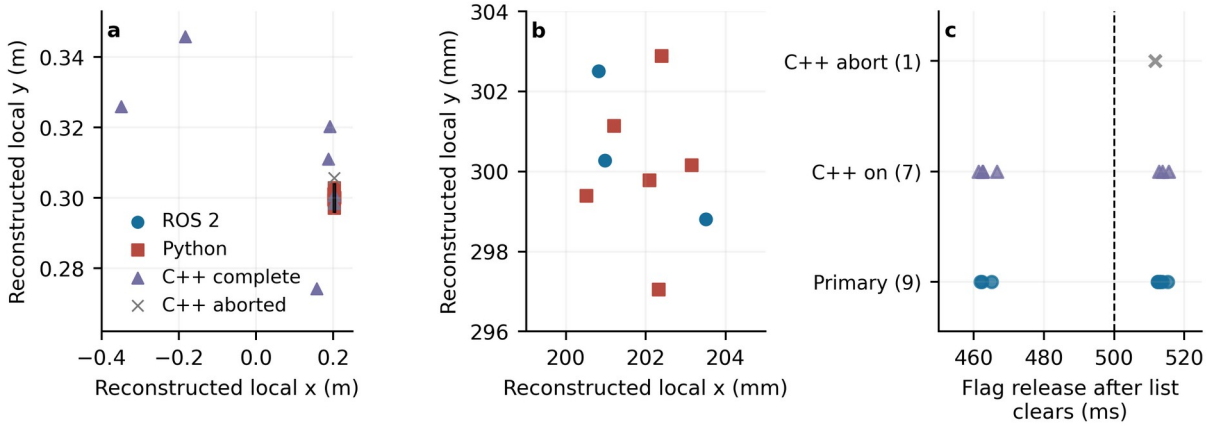


Figure 5. Conditional reconstruction and release timing. (a) All 18 rising-edge points from primary and secondary records, inverse-transformed with the same-frame pose using the audited planar conversion. The box marks the primary cluster enlarged in (b); these coordinates do not retain original point height. Earlier C++ points are more dispersed. (c) All 17 episodes with observed release, including one from the aborted session; the second aborted-session episode is censored and omitted here. The dashed line is the archived 500 ms hold constant, not an estimate fitted to the events. Arrival sampling and unavailable historical running-build equivalence limit interpretation.

4.7 What additional recordings can establish

The nine navigation archives contained 1,038 input commands and 1,038 forwarded commands. Ordinally paired planar velocity components matched exactly, but output-minus-input recording times ranged from −3.146 to 5.580 ms; three pairs were negative. Two negative pairs differed from both adjacent input vectors, while one repeated an adjacent body. Without unique identifiers or callback traces, the inversion source and exact causal pairing remain unresolved. These are recording differences, not negative processing latencies (Supplementary Table S9).

None of the nine archives recorded the avoidance State topic, and strict near-straight post-startup eligibility supplied only 1.301 s. The 20 collection summaries counted no ManualMoveCmd messages despite declaring that channel. Three fixed blocks from each of 66 further summary-positive files retained manual requests and State observations, but lacked known settings and complete incident windows. These cohorts constrain archive eligibility; they do not add matched avoidance trials or estimate failure prevalence (Supplementary Tables S9–S11).

4.8 Executable evidence reports

The workflow produced eight separate incident reports, preserving the aborted session's censoring and missing setting monitor. The four sessions with retained intervals remained unresolved; the other four required no attribution for the defined pause symptom. For the four primary records, 24 observation-availability configurations checked that withholding a required input suppressed its dependent assessment. These are software consistency checks on the same observations, not new robot trials or diagnostic-accuracy results. The reports, field contract and checklist accompany the offline package.

5 Discussion

5.1 What the records distinguish

The principal contribution is an auditable path from a motion symptom to bounded statements about the available evidence. In the worked case, queue accumulation occurred during smooth and interrupted records, while captured replies and feedback continued. The stale-feedback step required by the specified queue-growth explanation was not observed. This constrains that explanation within these windows without excluding other communication mechanisms or long-duration queue consequences.

Avoidance-flag rises preceded the retained near-zero intervals while nonzero requests continued. This supports control-state correspondence, not proof that avoidance logic caused the physical motion. Requested velocity, reported speed and a status flag do not expose the post-protection command or triggering decision. Occurrence through Python shows that the ROS 2 entry path was not necessary for these recorded associations. Shared platform behavior, changing environment and fixed order remain alternative explanations for the off/on contrast.

5.2 Implications for diagnostic instrumentation

The semantic audit narrows the remaining uncertainty. A plausible sensor identifier can be a default value, a displayed height can be inherited from localization, and a persistent flag can reflect release hold rather than sustained input. Collecting more instances of the same lossy output would not recover the missing information. The unresolved link is the source-tagged raw input and the controller decision for a particular event, with a verified mapping to the executing build.

For future logging, the case suggests retaining requested and post-protection velocity, source identity, the original point frame and height, an input timestamp, the last-hit time and the release condition on one inspectable timeline. This proposal follows the observed missing link; it has not been implemented or evaluated here. Middleware and operating-system tracing could further distinguish callback delay from network arrival.[6,7] The reversed recording timestamps in the separate archive illustrate why aligning message times alone cannot identify internal processing delay.

Excluding startup can remove ordinary acceleration intervals and also omit genuine reported flag transitions. Keeping symptom intervals and uncensored state transitions separate makes that tradeoff visible; neither is a count of physical faults.

5.3 Limitations and transferability

The incident comparison covers one platform and one date: four primary runs and four selected secondary sessions, with no randomized order, blinded observer, independent operator replication or fault-injection ground truth. The separate archive audit spans further source units and dates, but no added cohort supplies both a matched setting contrast and complete incident evidence. The conditions are not statistically independent replications of an avoidance treatment. Unrecorded obstacle geometry, floor interaction, localization and platform state may differ between runs. The case demonstrates execution of the assessment procedure. It does not establish diagnostic accuracy, field failure prevalence, safety performance or cross-platform causal transfer.

Odometry and State speed share a feedback origin, and there is no external pose tracker, wheel-current trace or synchronized video measurement. Accordingly, “pause” denotes the defined near-zero reported velocity. The operator notes corroborate visible hesitation but cannot calibrate its exact duration. Packet arrival times and sampled queues do not reveal internal scheduling or every short-lived saturation event. Parameter monitors constrain only their sampled instants. Library hashes establish limited artifact identity, not a complete historical executing version. Byte-matched disassembly supports interpretation of the archived path, not proof that the path ran during each event. Coordinate reconstruction and timer interpretation are therefore conditional explanations of reported fields. The archived parameter narrative alone is insufficient to assert the precise collision footprint, so no footprint classification is reported.

The investigation, archive selection and sensitivity choices are retrospective and informed by the incident. The roster audit is an accessible-file search rather than a representative fleet sample; inaccessible hosts and protected directories remain missing evidence. Evidence withholding cannot replace a prospective comparison against competing diagnostic systems. Public robot fault datasets have different sensing and fault labels, so adding them would not supply independent repetitions of this mechanism. A prospective study should repeat the setting contrast in randomized order on multiple days, preserve raw control inputs and use independent motion measurement. Such work would test transferability; its results are not assumed here.

6 Conclusion

We provide a reproducible workflow for cross-layer evidence auditing, demonstrated on four primary records and four separately analyzed C++ sessions. The primary records retained avoidance-state correspondence during continued upstream requests and feedback; the stale-feedback step of the specified queue-growth explanation was not observed. Artifact auditing constrained interpretation of identifiers, point height and release timing, while additional archives exposed eligibility and timestamp limits. The executable reports preserve missing fields and unresolved physical root cause. With one fixed-order run per primary condition, incomplete historical version linkage and unavailable triggering inputs, the study supports a documented diagnostic procedure and bounded observations, not causal fault isolation or validated performance on other robots.

Data and code availability

The accompanying reproducibility package contains deidentified derived records, analysis parameters, executable offline scripts, per-event results, the workflow input contract, executable evidence reports and figure source data. Network addresses, absolute site positions and identifying deployment records are excluded from that package. Original captures, deployed implementation artifacts and local provenance copies are retained in a separate private archive; their external release requires data-owner authorization. No public repository or unrestricted raw-data release is claimed.

References

1. Macenski S, Foote T, Gerkey B, et al. Robot Operating System 2: Design, architecture, and uses in the wild. Science Robotics 2022; 7(66): eabm6074. DOI: 10.1126/scirobotics.abm6074.

2. Pettersson O. Execution monitoring in robotics: A survey. Robotics and Autonomous Systems 2005; 53(2): 73-88. DOI: 10.1016/j.robot.2005.09.004.

3. Timperley CS, van der Hoorn G, Santos A, et al. ROBUST: 221 bugs in the Robot Operating System. Empirical Software Engineering 2024; 29(3): 57. DOI: 10.1007/s10664-024-10440-0.

4. Ferrando A, Cardoso RC, Fisher M, et al. ROSMonitoring: A Runtime Verification Framework for ROS. In: Mohammad A, Dong X and Russo M (eds) Towards Autonomous Robotic Systems. Lecture Notes in Computer Science, vol. 12228. Cham: Springer, 2020, pp.387-399. DOI: 10.1007/978-3-030-63486-5_40.

5. Caldas R, Piñera García JA, Schiopu M, et al. Runtime Verification and Field-Based Testing for ROS-Based Robotic Systems. IEEE Transactions on Software Engineering 2024; 50(10): 2544-2567. DOI: 10.1109/TSE.2024.3444697.

6. Bédard C, Lütkebohle I and Dagenais M. ros2_tracing: Multipurpose Low-Overhead Framework for Real-Time Tracing of ROS 2. IEEE Robotics and Automation Letters 2022; 7(3): 6511-6518. DOI: 10.1109/LRA.2022.3174346.

7. Bédard C, Lajoie PY, Beltrame G, et al. Message flow analysis with complex causal links for distributed ROS 2 systems. Robotics and Autonomous Systems 2023; 161: 104361. DOI: 10.1016/j.robot.2022.104361.

8. Saito S and Jiang C. Fault-tolerant navigation framework for omnidirectional mobile robots under actuator failures via an enhanced potential field. International Journal of Advanced Robotic Systems 2026; 23(1): 17298806251408590. DOI: 10.1177/17298806251408590.

9. Runeson P and Höst M. Guidelines for conducting and reporting case study research in software engineering. Empirical Software Engineering 2009; 14(2): 131-164. DOI: 10.1007/s10664-008-9102-8.

10. Foote T. REP 107: Diagnostic System for Robots Running ROS. ROS Enhancement Proposals, 2010. https://docs.ros.org/en/independent/api/rep/html/rep-0107.html (accessed 19 October 2026).